```
mixed(1, 2, 3)    # => [1, 2, [3]]
mixed(1, 2, 3, 4) # => [1, 2, [3, 4]]
```

Keyword Arguments

Ruby methods may declare keyword arguments using the syntax name: _default_value_ for each. These arguments must follow any regular arguments in the list. The default value is optional, in which case you just type name:.

reference/kwargs.rb

```
def header(name, level:, upper: false)
  name = name.upcase if upper
  "<h#{level}>#{name}</h#{level}>"
end

header("Introduction", level: 1)             # => "<h1>Introduction</h1>"
header("Getting started", level:2)           # => "<h2>Getting started</h2>"
header("Conclusion", upper: true, level: 1)  # => "<h1>CONCLUSION</h1>"
```

When calling a method with keyword arguments, the keyword arguments do not need to be in the same order in the method call as they are in the method definition. If you call a method that has keyword arguments, you must provide values for each keyword that does not have a default value. If you do not provide a value for a keyword with a default, the default will be used.

If you pass keyword parameters that are not defined as arguments, an error will be raised unless you also define a double splat argument, **_arg_. The double splat argument will be defined as a hash containing any uncollected keyword parameters passed to the method.

reference/kwargs_2.rb

```
def header(name, level: 1, upper: false, **attrs)
  name = name.upcase if upper
  attr_string = attrs.map { |k, v| %(#{k}="#{v}") }.join(" ")
  "<h#{level} #{attr_string}>#{name}</h#{level}>"
end

header("TOC", class: "nav", level: 2, id: 123)
```

Block Argument

The optional block argument must be the last in the list. Whenever the method is called, Ruby checks for an associated block. If a block is present, it is converted to an object of class Proc and assigned to the block argument. If no block is present, the argument is set to nil.

```
def example(&block)
puts block.inspect
end

example
example { "a block" }
```

produces:

```
nil
#<Proc:0x0000000104516878 prog.rb:6>
```